

KitchenFlow

Premium Kitchen Operations Dashboard — Product Case Study

OVERVIEW

KitchenFlow is a premium restaurant kitchen operations dashboard built as a portfolio-grade product UI for high-volume restaurants. It gives chefs, expeditors, and managers a way to track live tickets, station load, service health, and order details without visual clutter — the kind of interface clarity a product manager gets from a SaaS dashboard, brought to a kitchen.

Role: Sole frontend engineer — designed and built the entire product end-to-end (architecture, UI, state, motion).

TECH STACK

- Next.js (App Router)
- React 19
- TypeScript
- Tailwind CSS v4
- lucide-react (icons)
- Client-side state via useState / useMemo — no backend; all service data is realistic mock data simulating a live dinner service, by design (this is a frontend engineering showcase, not a connected POS system)

PRODUCT MODULES

Six modules, all driven by one shared live service state:

Command — Live kitchen ticket board across four lanes: New, Fire, Plating, Ready. Includes priority/station/search filters and a focused ticket detail panel (items, allergies, notes, timing, one-click progression).

Intake — New order entry into the live queue.

Prep — Station-level prep tracking.

Inventory — Ingredient and stock visibility tied to the live service.

Insights — Service analytics: completed tickets, held tickets, average ticket time, dinner-service target progress, promised-time risk, and allergy-sensitive order tracking.

Crew — Station controls (Grill, Saute, Garde Manger, Pastry) with live load percentages, and a chef board showing station ownership and readiness.

DESIGN APPROACH

Dark, premium UI with a gold accent color, built around a calm, low-clutter visual language so operational information stays scannable under pressure. Motion is deliberately lightweight — opacity and transform transitions only — so the interface stays fast and doesn't distract from the task of running service.

ENGINEERING NOTES

Single-page Next.js App Router application (~1,100 lines in the main page component) using React state (useState, useMemo) to drive all six modules from one shared source of truth, so ticket counts, station load, and crew status stay consistent across every view without prop drilling or a state library.

LINKS

GitHub: github.com/Hritwik001/KitchenFlow

A recorded product walkthrough is included as a live demo video in the portfolio case study.

Hritwik Tiwary — Frontend Engineer (React / Next.js)